Infrastructure & Pipeline

Architecture of the ipecho service - from a git push to a geolocated request on the map

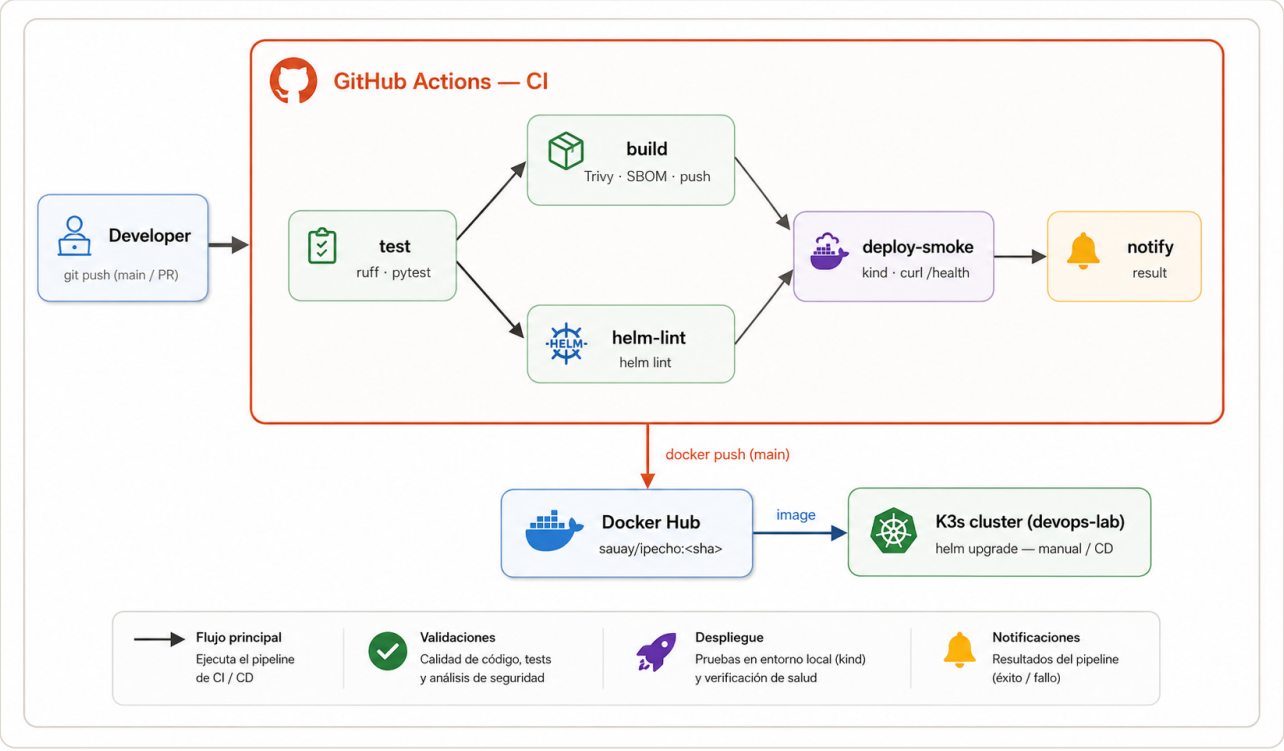
Carlos Pereyra

July 20, 2026

This document gives a visual overview of the ipecho lab: how a change flows through continuous integration to a published container image, and how a request travels from a browser through Cloudflare into the K3s cluster and back. Two diagrams are provided - the CI/CD pipeline and the runtime infrastructure - each followed by a short walkthrough.

FIELD	VALUE
Project	lab-devops / ipecho
Repository	github.com/pereyra-carlos/devops-lab
Registry	Docker Hub - sauay/ipecho
Cluster	K3s (nodes 192.168.0.51 / .52), namespace devops-lab
Public entry	Cloudflare tunnel cf-infra
Public URLs	devops-lab.pereyra.ar (app), devops-lab-grafana.pereyra.ar (dashboard)

1. CI/CD pipeline

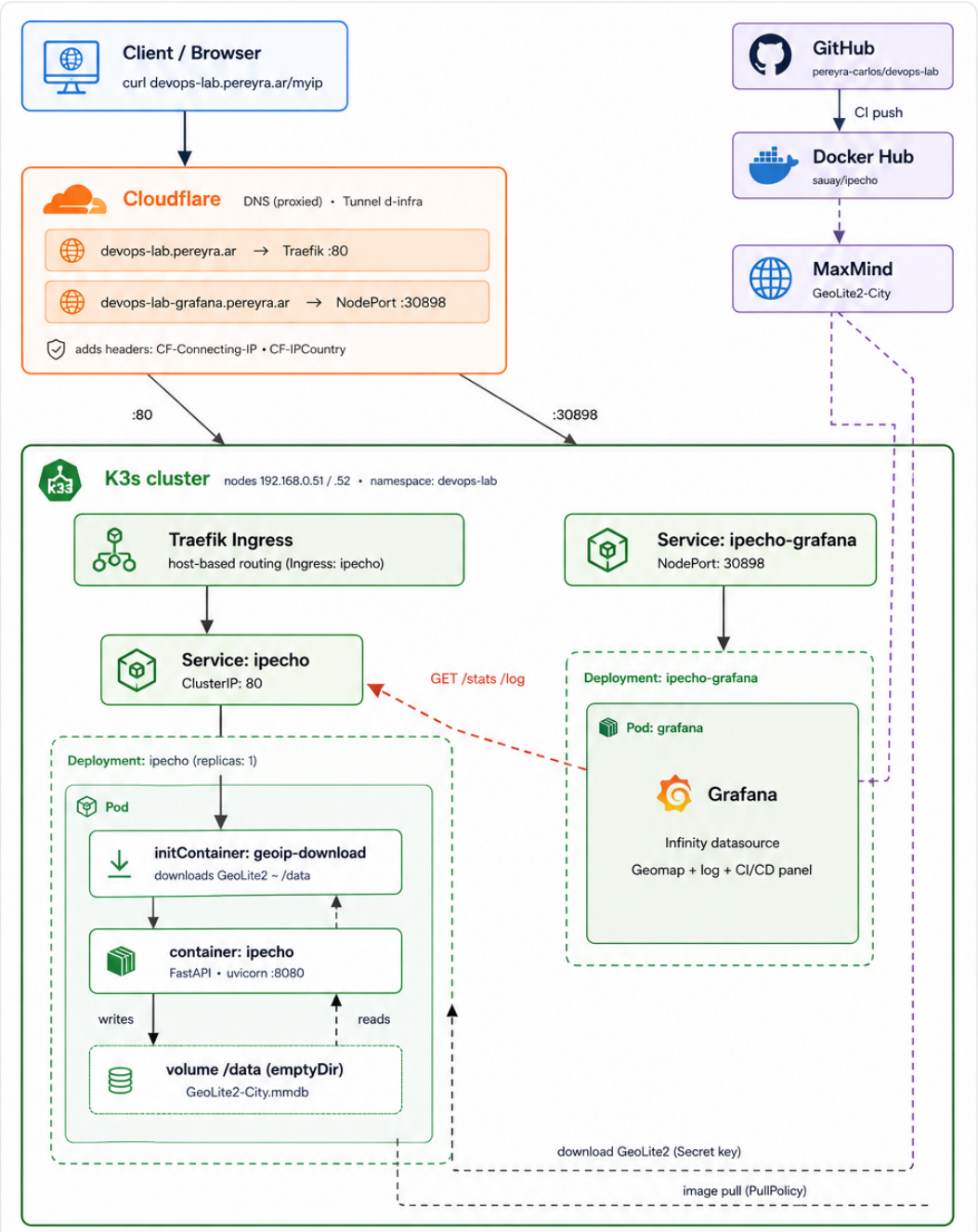


A push to main (or any pull request) triggers the GitHub Actions pipeline:

1. test runs first - ruff (lint) and pytest (unit tests). It gates everything else.
2. build produces the image, scans it with Trivy (failing on CRITICAL), emits a CycloneDX SBOM, and pushes to Docker Hub - but only on push events, tagged with the commit SHA.
3. helm-lint validates the chart in parallel with build.
4. deploy-smoke depends on both: it spins up an ephemeral kind cluster, installs the chart with Helm, and curls /health and /ready. This proves the artifact actually runs.
5. notify reports the aggregate result.

Delivery to the long-lived K3s cluster is a separate step (helm upgrade), which is why the version running in production can lag the latest built image. The natural next step is GitOps (for example Argo CD) so the cluster reconciles to the repository automatically.

2. Runtime infrastructure



Leyenda

- Flujo de tráfico
- Flujo interno
- Conexión externa

Componentes

- Cliente
- Edge (Cloudflare)
- K3s / Kuberentes
- Servicios externos

Puertos

- :80 HTTP (Traefik → ipecho)
- :30898 NodePort (ipecho-grafana)

Notas

- Cloudflare agrega CF-Connecting-IP y CF-IPCountry.
- Grafana usa MaxMind GeoLite2-City para Geomap.
- Los datos GeoLite2 se descargan al inicio via initContainer.

Request lifecycle

1. A client calls https://devops-lab.pereyra.ar/myip.
2. Cloudflare (DNS proxied) forwards the request through the cf-infra tunnel and injects CF-Connecting-IP (the real client IP) and CF-IPCountry.
3. For the app host, the tunnel targets Traefik on port 80. Traefik's host-based Ingress routes to the ipecho ClusterIP Service, which forwards to the pod.
4. The application reads CF-Connecting-IP and geolocates it using the GeoLite2 database that the init container downloaded from MaxMind (with a license key from a Kubernetes Secret) into the shared /data volume.
5. The Grafana host targets the grafana NodePort (30898) directly, bypassing Traefik. The dashboard queries the application's /stats and /log endpoints through the Infinity datasource to draw the map and the log table.
6. Docker Hub supplies the container image (pulled by the deployment); GitHub Actions is what pushed it there.

COMPONENT	ROLE
Cloudflare (cf-infra tunnel)	Public DNS + TLS, injects real client IP headers
Traefik Ingress	Host-based routing to the app Service
Service: ipecho (ClusterIP)	Stable in-cluster address for the app pod
Deployment: ipecho (1 replica)	Runs the FastAPI app; state is in-memory
initContainer: geoip-download	Fetches GeoLite2 into a shared volume at pod start
Service: ipecho-grafana (NodePort)	Exposes Grafana on 30898
Grafana	Reads app JSON via Infinity; renders the map dashboard

> The whole path is reproducible from the repository: the chart and manifests describe the cluster objects, the workflow describes the pipeline, and the Grafana datasource and dashboard are provisioned as code.